Traffic Sign Recognition

1. Course Content

Basic: Start the example program, use traffic sign recognition alone, and make corresponding response actions.

Advanced: Understand the traffic sign recognition action trigger flow and extension action methods.

2. Traffic Signs

2.1 Types of Traffic Signs

- Straight, Right Turn, Honking, Sidewalk

- Speed Limit, Stop, School, Parking B

- Traffic Light, Parking A

2.2 Traffic Sign Response Actions

- Straight
  - Drive according to the set speed, or resume driving after stopping at a STOP sign
- Right Turn
  - Trigger the fixed right turn action
- Honking
  - Trigger car horn sound effect
- Sidewalk
  - Trigger car horn sound effect, decelerate for 1 second
- Speed Limit
  - Car decelerates for 3 seconds, then resumes normal speed
- School
  - Trigger car horn sound effect, decelerate for 1 second
- Parking A, B
  - Trigger preset fixed parking action
- Traffic Light
  - Red Light
    - When not combined with road network planning: trigger deactivation of lane-keeping autonomous driving state
    - When combined with road network planning: cancel the currently executing navigation task
  - Green Light
    - When not combined with road network planning: trigger activation of lane-keeping autonomous driving state

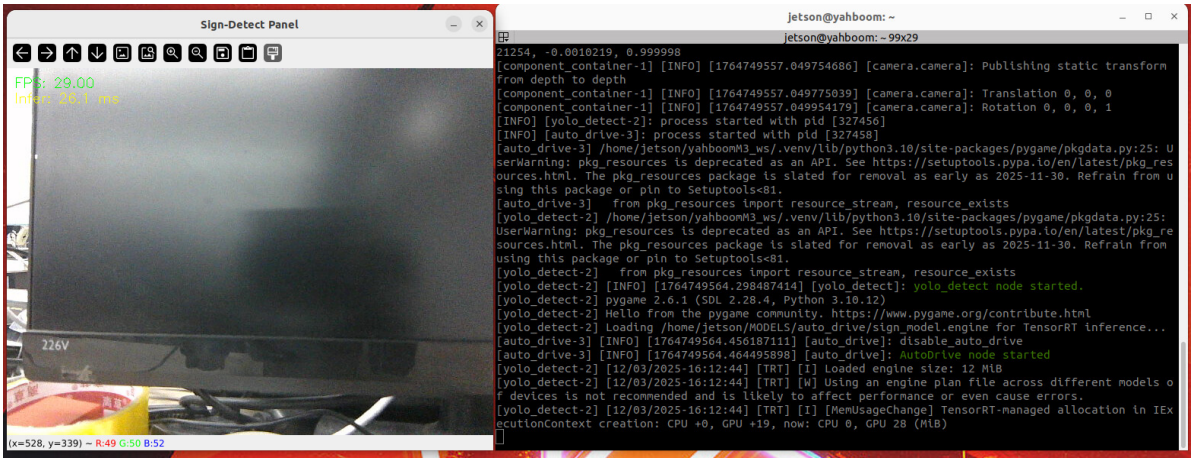
- When combined with road network planning: resume to the previous navigation task

3. Run Example Program

- Start autonomous driving program

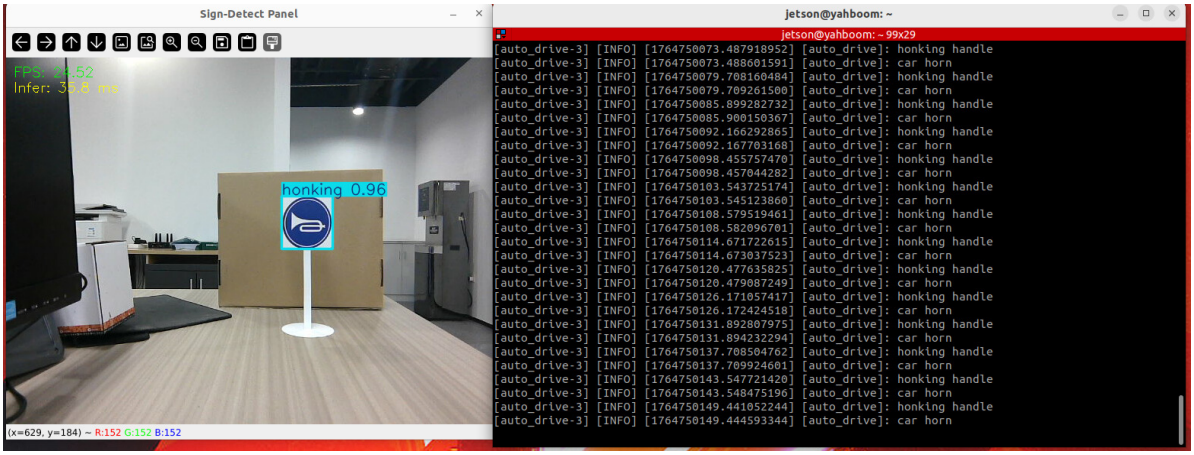
```
ros2 launch m3_bringup auto_drive.launch.py unkeeplane:=True
```

> [!TIP]
>
> The unkeeplane startup parameter here is used to disable the lane line detection model and processing thread

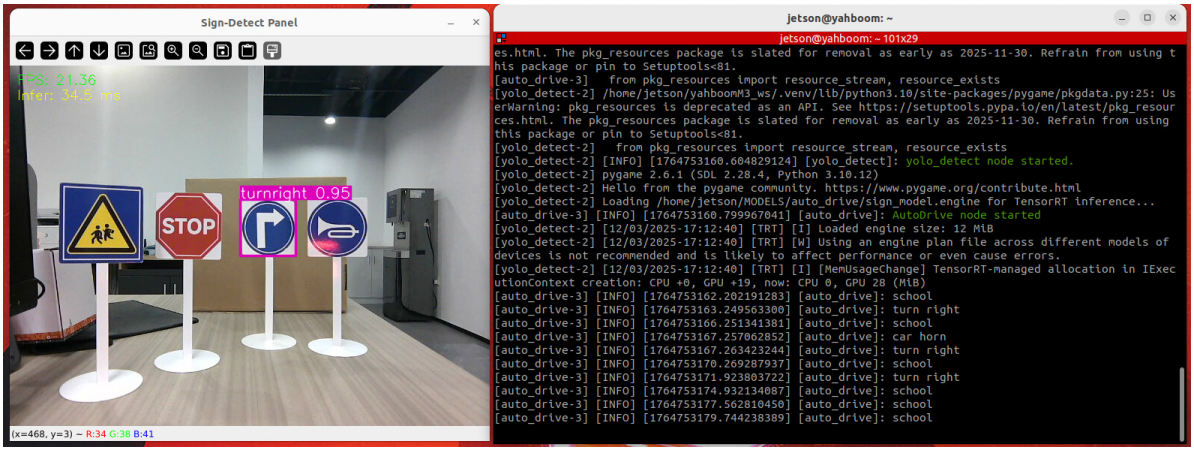


- Here we use the honking sign as an example for testing. The terminal will print the currently executed response action

> [!NOTE]
>
> - Action responses have a cooldown time to ensure that the same action is not triggered frequently within a short period. The cooldown time adjustment will be explained in the debugging special topic.



- Other traffic signs all trigger corresponding action functions, which can be tested one by one.



4. Source Code Analysis

4.1 Traffic Sign YOLO Inference Program

- The main logic source code is located in `yolo_detect.py` of the `auto_drive` package:

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/yolo_detect.py
```

- YOLOv11 inference source code:

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/utils/yolov11_infer.py
```

- `Sign_Detect` is a subclass that inherits from `YOLO_MODEL`, containing various methods for traffic sign detection

```
class Sign_Detect(YOLO_MODEL):
    """Road sign inspection"""
    def __init__(self,config_file_path):
        with open(config_file_path, "r") as file:
            full_config = yaml.safe_load(file)
            self.config_param = full_config.get("sign_detection", {})

    def init_yolo_engine(self):
        self.image_queue = queue.Queue(maxsize=10)
        #Obtain the reasoning parameters
        self.conf=self.config_param.get("conf",0.25)
        self.iou=self.config_param.get("iou",0.75)
        self.rect=self.config_param.get("rect",True)
        self.half=self.config_param.get("half",True)
        self.batch=self.config_param.get("batch",1)
        self.max_det=self.config_param.get("max_det")
        self.augment=self.config_param.get("augment",False)
        self.verbose=self.config_param.get("verbose",True)
        self.classes=self.config_param.get("classes",None)
        self.device=self.config_param.get("device", "cuda:0")

        try:
            self.yolo_model =
YOLO(self.config_param.get("model_path"),task="detect")
```

```
        except:
            return False
        return True

    def put_image(self,image):
        '''Add images to the processing queue'''
        try:
            self.image_queue.put_nowait(image)
        except queue.Full:
            return

    def clear_queue(self):
        '''Clear the queue'''
        while not self.image_queue.empty():
            self.image_queue.get()

    def get_infer_result(self,source,stream:bool=True):
        '''Obtain the reasoning result of a single frame image'''
        result = self.yolo_model(source,
                                 stream=stream,
                                 conf=self.conf,
                                 iou=self.iou,
                                 rect=self.rect,
                                 half=self.half,
                                 device=self.device,
                                 batch=self.batch,
                                 max_det=self.max_det,
                                 augment=self.augment,
                                 verbose=self.verbose,
                                 classes=self.classes
                                 )
        return result
```

- The `image_callback` callback function is responsible for continuously subscribing to the camera image topic and putting it into the `sign_detect` processing queue
- The `handle_sign` thread is responsible for taking video frames from the sign_detect queue and using the `get_infer_result` method for YOLO inference
- The YOLO recognition results are published to the `/sign_detect` topic through the `self.sign_pub_counter` publisher, and then the `auto_drive` node subscribes to this topic and executes corresponding actions.

```
    def image_callback(self, msg: Image):
        '''The ROS image callback function converts the ROS image to OpenCV
format and adds it to the processing queue.
        '''
        cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
        self.sign_detect.put_image(cv_image.copy())

    def handle_sign(self):
        '''Process the road sign detection results in the image'''
        while True:
            try:
                frame = self.sign_detect.image_queue.get(timeout=0.01)
            except queue.Empty:
```

```
                continue
            self.sign_pub_counter += 1
            infer_start = time.time()
            results=self.sign_detect.get_infer_result(frame,True)

            for res in results:
                if res.boxes is not None:
                    annotated_frame = res.plot()
                    boxes = res.boxes
                    for i in range(len(boxes)):
                        box_coords = boxes.xyxy[i].tolist()# Get the bounding
box coordinates (xyxy format: xmin, ymin, xmax, ymax)
                        class_name = res.names[int(boxes.cls[i].item())]# Get
the object name
                        confidence = boxes.conf[i].item()# Get the confidence
                        if self.sign_pub_counter % 15 == 0:
                            self.sign_publisher.publish(DetectionObject(
                                class_name=class_name,
                                confidence=confidence,
                                xmin=float(box_coords[0]),
                                ymin=float(box_coords[1]),
                                xmax=float(box_coords[2]),
                                ymax=float(box_coords[3])
                            ))
                            self.sign_pub_counter=0

            infer_end = time.time()
            self.frame_count += 1
            curr_time = time.time()
            time_diff = curr_time - self.prev_time
            if time_diff >= 1.0:
                self.fps = self.frame_count / time_diff
                self.frame_count = 0
                self.prev_time = curr_time

            cv2.putText(annotated_frame,f"FPS: {self.fps:.2f}",(10,
25),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),1,)
            infer_time = (infer_end - infer_start) * 1000
            cv2.putText(annotated_frame, f"Infer: {infer_time:.1f} ms", (10,
45),cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 1)

            cv2.imshow("Sign-Detect Panel", annotated_frame)
            cv2.waitKey(1)
```

4.2 Action Execution Program

- The program source code is located in the `auto_drive.py` program of the `auto_drive` package

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/auto_drive.py
```

- In the `auto_drive` node, the `sign_callback` callback function subscribes to the `/sign_detect` topic, and then executes actions corresponding to traffic signs in the handle_events thread

```
    def handle_events(self):
        '''Handle target recognition events'''
        while True:
            try:
                object:Object=self.event_queue.get(timeout=0.2)
            except queue.Empty:
                continue
            if self.__check_cooldown(object.class_name):
                continue
            # self.get_logger().info(f'{object.class_name} handle')

            if object.class_name=="straight" and self.sign_enable_list[10]:
                self.go_straight()
            elif object.class_name=="turnright" and self.sign_enable_list[11]:
                self.turn('right')
            elif object.class_name=="turnright_ground":
                pass
            elif object.class_name=="honking" and self.sign_enable_list[1]:
                self.car_horn()
            elif object.class_name== "sidewalk" and self.sign_enable_list[6]:
                self.sidewalk()
            elif object.class_name=="sidewalk_ground" and
self.sign_enable_list[7]:
                pass
            elif object.class_name=="speedlimit" and self.sign_enable_list[8]:
                self.speedlimit()
            elif object.class_name=="stop" and self.sign_enable_list[9]:
                self.stop()
            elif object.class_name=="school" and self.sign_enable_list[5]:
                self.school()
            elif object.class_name=="parkA" and self.sign_enable_list[2]:
                    self.parking_A()
            elif object.class_name=="parkB" and self.sign_enable_list[3]:
                self.parking_A()
            elif object.class_name=="greenlight" and self.sign_enable_list[0]:
                self.greenlight()
            elif object.class_name =="redlight" and self.sign_enable_list[4]:
                self.redlight()
            elif object.class_name=="yellowlight" and self.sign_enable_list[13]:
                pass
            elif object.class_name=="out_park" :
                self.out_parking()
```

- The `__check_cooldown` method is used to check whether the current action to be executed is still within the cooldown time. If it is still within the cooldown time, skip this action; otherwise execute the action

```
    def __check_cooldown(self,class_name:str)->bool:
```

```
        '''Check whether the target recognition event is repeatedly triggered
within a short period of time during the cooldown period.
        '''

        current_time = time.time()
        if class_name in self.last_processed_time:

            time_since_last = current_time -
self.last_processed_time[class_name]
            if time_since_last < self.cooldown_time:
                return True
            else:
                self.last_processed_time[class_name] = current_time
                return False
        else:
            self.last_processed_time[class_name] = current_time
            return False
```

- Action functions corresponding to other traffic signs are as follows:

```
    def handle_light(self):
        '''Asynchronous processing of lighting events'''
        while True:
            try:
                light_event = self.light_queue.get(timeout=0.5)
            except queue.Empty:
                continue
            if light_event[0]==107.0:#Ambient light red
                self.current_rgb_light.a=0.0
                self.current_rgb_light.r=100.0
            elif light_event[0]==108.0:#Ambient light yellow
                self.get_logger().info('turn yellow')
                self.current_rgb_light.r=100.0
                self.current_rgb_light.g=100.0
            elif light_event[0]==100.0:#Turn off the lights
                self.current_rgb_light.a=100.0
            else:
                self.current_rgb_light.a=light_event[0]

            if light_event[1]!=0.0:
                time.sleep(light_event[1])
                self.current_rgb_light.a=100.0
            self.rgb_pub.publish(self.current_rgb_light)

    def speedlimit(self):
        '''speedlimit'''
        self.get_logger().info("speedlimit")
        self.enable_auto_drive(False)
        self.__set_current_speed(linear_x=0.09)
        time.sleep(3.0)
        self.enable_auto_drive(True)

    def turn(self,direction):
        '''Turn'''
```

```
        if self.R == 0:#Turn in place
            if direction == 'left':
                self.get_logger().info('turn left')
                self.__set_current_speed(angular_z=self.angular_speed )
            elif direction == 'right':
                self.get_logger().info('turn right')
                self.__set_current_speed(angular_z=-self.angular_speed  )
        else:#Turn with radius
            self.current_linear_x=self.drive_speed
            if direction == 'left':
                self.get_logger().info('turn left')
                self.__set_current_speed(linear_x=self.drive_speed,
angular_z=self.drive_speed / self.R )
            elif direction == 'right':
                self.get_logger().info('turn right')
                self.__set_current_speed(linear_x=self.drive_speed, angular_z=-
self.drive_speed / self.R )
        time.sleep(self.duration)
        self.__set_current_speed(linear_x=self.drive_speed, angular_z=0.0)

    def go_straight(self):
        '''go straight'''
        self.get_logger().info('go straight')
        self.__set_current_speed(self.drive_speed)
        self._light_control(100.0)

    def stop(self):
        '''stop'''
        self.get_logger().info('stop')
        if self.conbine_nav:
            self.cancel_nav_pub.publish(Bool(data=True))
        else:
            self.enable_auto_drive(False)
            self.__set_current_speed(linear_x=0.0)
        self._light_control(107.0)


    def sidewalk(self):
        '''sidewalk'''
        self.get_logger().info('sidewalk')
        self.play_audio(self.honk_audio_path)
        self.__set_current_speed(linear_x=0.09)
        time.sleep(1.0)

    def redlight(self):
        '''redlight'''
        self.get_logger().info('redlight')
        if self.conbine_nav:
            self.cancel_nav_pub.publish(Bool(data=True))
        else:
            self.enable_auto_drive(False)
            self.__set_current_speed()

    def greenlight(self):
        '''greenlight'''
        self.get_logger().info('greenlight')
        if self.conbine_nav and self.recentest_pose is not None:
            self.road_net_nav_pub.publish(self.recentest_pose)
```

```
        else:
            self.enable_auto_drive(True)

    def school(self):
        '''School'''
        self.get_logger().info('school')
        self.play_audio(self.honk_audio_path)
        self.__set_current_speed(linear_x=0.09)
        time.sleep(1.0)

    def car_horn(self):
        '''trumpet'''
        self.get_logger().info('car horn')
        self.play_audio(self.honk_audio_path)

    def slow_down(self):
        '''slow down'''
        self.get_logger().info('slow down')
        self.__set_current_speed(0.1)
        self._light_control(108.0,3.0)#Yellow light prompt for 3 seconds
        time.sleep(3.0)
        self.__set_current_speed(self.drive_speed)

    def parking_A(self):
        ''''Parking A, B'''
        self.get_logger().info('parking A')
        time.sleep(2.5)
        self.__set_current_speed()
        time.sleep(3.0)
        self.__set_current_speed(linear_y=0.3)
        time.sleep(3.0)
        self.__set_current_speed()
    def parking_B(self):
        '''Parking B'''
        pass

    def out_parking(self):
        '''Vehicle leaving the depot'''
        pass

    def
__set_current_speed(self,linear_x:float=0.0,linear_y:float=0.0,angular_z:float=0
.0)->None:
        '''Set current vehicle speed'''
        self.current_speed.linear.x=linear_x
        self.current_speed.linear.y=linear_y
        self.current_speed.angular.z=angular_z
        self.cmd_vel_pub.publish(self.current_speed)
    def _light_control(self,color:float,time:float=0.0):
        '''Lighting control'''
        self.light_queue.put([color,time])

    def __enable_sign(self,number):
        '''Enable specified flags to take effect'''
        self.sign_enable_list[number]=True

    def __disable_sign(self,number):
```

```
        '''The specified flag is invalid.'''
        self.sign_enable_list[number]=False

    def alarm(self,mode):
        '''Lighting and sound alarm / Alarm action '''
        self.get_logger().info(f'alarm,{mode}')

        if mode:
            self._light_control(101.0)
            self.play_audio(self.alarm_audio_path)
        else:
            self._light_control(100.0)
            try:
                if pygame.mixer.get_init() and pygame.mixer.music.get_busy():
                    pygame.mixer.music.stop()
                    pygame.mixer.quit()
            except pygame.error:
                pass

    def play_audio(self,file_path: str) -> None:
        pygame.mixer.init()
        pygame.mixer.music.load(file_path)
        pygame.mixer.music.play()
```

5. View Communication Graph Between Nodes

```
ros2 run rqt_graph rqt_graph
```